Questions on Unit 03

The following multiple-choice questions are designed to assess your knowledge of this unit's section. See the version without solutions first. Each question is worth 1.0 points, with the score being divided among the correct answers where applicable. You should aim to complete each question within an average time of three to four minutes. You may solve the questions in any order you prefer; once finished, compare your responses with the provided solution file. You have successfully passed the assessment if you achieve at least 50% of the available points.

Ex. In C/C++ projects
Choose one or more options

- [x] A C++ source file typically uses the extension .cpp.
- [x] Header files may use extensions such as .h or .hpp.
- [ ] A C++ project normally consists only of a single .cpp file.
- [x] Separating declarations in headers and definitions in .cpp files can help create libraries.
- [ ] C++ is almost always implemented as an interpreted language.

Ex. IO streams and basic console IO
Choose one or more options

- [x] std::cout is the standard output stream in C++.
- [x] std::cin is the standard input stream in C++.
- [ ] std::cerr is buffered in the same way as std::clog.
- [x] std::endl inserts a newline and flushes the output buffer.
- [ ] The C stdio functions printf and scanf are safer than C++ streams by default.

Ex. Namespaces and using
Choose one or more options

- [x] A namespace groups identifiers to avoid name collisions.
- [x] std is the namespace that contains the standard C++ library facilities.
- [x] using namespace std; makes all std names visible without qualification.
- [ ] It is recommended to put using namespace std; in header files.
- [x] using std::cout; introduces only cout from namespace std.

Ex. Basic arithmetic and literals
Choose one or more options

- [x] Integer literals can be written in decimal, octal, or hexadecimal in C++.
- [ ] A floating-point literal must always use an exponent part (e.g., 1.0e3).
- [x] Character literals are written inside single quotes, e.g., 'a'.
- [ ] String literals are written inside single quotes, e.g., "text".
- [x] Mixing signed and unsigned integers can lead to surprising results in expressions.

Ex. Fundamental types and qualifiers
Choose one or more options

- [x] The type bool can hold the values 1 and 0.
- [x] A const object must be initialized at the point of definition.
- [x] A volatile object tells the compiler that its value can change outside the program's control.
- [ ] A const volatile variable cannot be read by the program.
- [ ] The qualifiers const and volatile cannot be combined on the same object.

Ex. Pointers
Choose one or more options

- [x] A pointer stores the address of another object.
- [ ] The size of a pointer depends on the type it points to.
- [x] The null pointer literal in modern C++ is nullptr.
- [x] It is good practice to initialize a pointer to nullptr if it doesn't point to a valid object yet.
- [ ] Pointer arithmetic is allowed on all pointer types, including void*.

Ex. References
Choose one or more options

- [x] A reference is an alternative name (alias) for an existing object.
- [ ] A reference does not have to be initialized when it is defined.
- [ ] After initialization, a reference can be reseated to refer to a different object.
- [x] A reference cannot be bound directly to a literal.
- [x] In typical implementations, references are often realized internally using pointers.

Ex. const and pointers
Choose one or more options

- [ ] A "top-level const" applies to the object pointed itself.
- [ ] A "low-level const" applies to the pointer object to.
- [x] A pointer to const int prevents changing the int through that pointer.
- [x] A const pointer to int cannot change the address it stores.
- [ ] A const pointer to const int cannot change either the address or the pointed value.

Ex. volatile and hardware interaction
Choose one or more options

- [x] volatile prevents certain compiler optimizations on accesses to the variable.
- [x] volatile is often used for memory-mapped I/O registers.
- [ ] volatile alone provides full thread synchronization and atomicity in C++20.
- [x] A const volatile object may be changed by hardware but not by C++ code.
- [ ] volatile guarantees that no data race can occur in a multithreaded program.

Ex. auto and decltype
Choose one or more options

- [x] The keyword auto lets the compiler deduce the type of a variable from its initializer.
- [x] A variable declared with auto must have an initializer.
- [ ] In a single declaration using auto, different variables can have different deduced type.
- [x] decltype yields the type of an expression without evaluating that expression.
- [ ] decltype can be used only with fundamental types.

Ex. Control statements
Choose one or more options

- [x] C++ supports if, if-else, and switch for conditional execution.
- [x] The while and do-while statements are both used for loops.
- [x] The for statement in C++11 can iterate directly over elements of a container.
- [ ] The break statement does not exits in C++.
- [ ] The continue statement terminates the entire program.

Ex. Exceptions (basic exampl
Choose one or more options

- [x] throw is used to signal that an exceptional condition has occurred.
- [x] try blocks contain code that might throw exceptions.

- [x] catch blocks handle exceptions of specific types.
- [x] A catch(...) handler can catch exceptions of any type.
- [ ] Exceptions are the only way to report errors in C++.

Ex. Old-style casts and static_cast
Choose one or more options
- [ ] static_cast can safely cast between completely unrelated pointer types.
- [x] A C-style cast can behave like static_cast, const_cast, or reinterpret_cast depending on context.
- [x] static_cast is typically used to convert between related arithmetic types.
- [x] static_cast performs compile-time checks on the validity of the conversion.
- [ ] Using static_cast to narrow a value (e.g., double to int) cannot lose information.

Ex. const_cast
Choose one or more options
- [x] const_cast can remove const qualification from a pointer or reference type.
- [x] Writing through a pointer obtained by const_cast from a truly const object has undefined behavior.
- [ ] const_cast can add const to a type but cannot remove it.
- [x] const_cast is often used in the implementation of function overloads that differ by const.
- [ ] const_cast can change the underlying type of the object.

Ex. reinterpret_cast
Choose one or more options
- [x] reinterpret_cast is intended for low-level, implementation-dependent conversions.
- [x] reinterpret_cast can be used to view the bit pattern of a float as an int.
- [ ] reinterpret_cast is generally safer than static_cast for arithmetic types.
- [ ] Misusing reinterpret_cast cannnot lead to undefined behavior.
- [ ] reinterpret_cast is equaivalent to using unions in C for type punning.

Ex. Function basics
Choose one or more options
- [x] A C++ function has a return type, a name, and a parameter list.
- [x] A function that does not return a value should be declared with return type void.
- [ ] In C the main may have parameters such as int argc, char* argv[] but not in C++.
- [ ] Every function must take at least one parameter.
- [x] A function declared but never defined can cause linker errors if it is called.

Ex. Parameter passing (value, pointer, referenc
Choose one or more options
- [x] Passing by value copies the argument into the function parameter.
- [ ] Changes to a parameter passed by value affect the caller's variable.
- [x] Passing a pointer by value allows the function to modify the object pointed to.
- [ ] A reference parameter must be checked for not being nullptr.
- [x] Passing by reference allows the function to operate directly on the caller's variable.
- [ ] A reference parameter can be bound to nullptr.

Ex. Common pitfalls in swapping
Choose one or more options
- [x] A swap function that takes arguments by value will not swap the caller's variables.
- [x] To swap caller variables, swap should use reference parameters.

- [x] Swapping via pointers requires dereferencing inside the function.
- [ ] Swapping via copies is always more efficient than using references.
- [x] The standard library provides std::swap for general use.

Ex. Constant parameters in functions
Choose one or more options
- [x] Parameters that a function does not modify should be declared const when passed by reference.
- [x] Declaring a parameter const makes it illegal to modify that parameter inside the function.
- [x] const parameters can help the compiler catch unintended modifications.
- [ ] const parameters change the calling syntax at the call site.
- [x] A const reference parameter can bind to temporaries and large objects efficiently.

Ex. Functions with varying parameters
Choose one or more options
- [x] C++ supports parameter packs via templates for variadic functions.
- [x] std::initializer_list can be used when all varying arguments share the same type.
- [x] The ellipsis (...) syntax supports C-style variadic functions.
- [ ] Variadic functions using ellipsis perform compile-time type checking for all arguments.
- [x] Variadic functions need special handling to process their arguments.

Ex. Function overloading
Choose one or more options
- [x] In C++, multiple functions can have the same name in the same scope.
- [ ] Overloaded functions cannot differ in their parameter list.
- [ ] Overloading based only on different return types is allowed.
- [x] The compiler selects the best-matching overload at compile time.
- [x] Overloading can simplify APIs by avoiding many different function names.

Ex. Default arguments
Choose one or more options
- [x] A parameter with a default argument may be omitted at the call site.
- [ ] If a parameter has a default value, all previous parameters must also have default values.
- [x] Default arguments are typically specified in function declarations.
- [ ] The same parameter can be given different default values in the same scope.
- [x] Default arguments are bound at compile time, not at runtime.

Ex. Pointers to functions (conceptual)
Choose one or more options
- [ ] A function pointer can point to objects as well as functions.
- [x] The name of a function can decay to a pointer to that function.
- [x] A function pointer can be passed as an argument to another function.
- [x] A function pointer holds the address of executable code.
- [x] A function pointer can be dereferenced and called using the same parameter list as the function type.

Ex. Classes and basic concepts
Choose one or more options
- [x] A class defines a new type that bundles data and related operations.
- [x] Member functions are also called methods.
- [ ] In C++, a class can be defined only with the keyword class.

- ☐ A class can only contain data members, not functions.
- ☑ One goal is to make user-defined types behave similarly to built-in types.

Ex. Struct vs class and access
Choose one or more options
- ☐ For a struct, members are private by default.
- ☐ For a class, members are public by default.
- ☑ Access specifiers include public, private, and protected.
- ☐ Changing an access specifier changes the memory footprint of the object.
- ☑ public members are accessible from any part of the program with access to the object.

Ex. Encapsulation and access control
Choose one or more options
- ☑ Encapsulation groups data and methods into a single unit.
- ☑ Encapsulation helps separate interface from implementation.
- ☐ Good practice is to make data members public whenever possible.
- ☐ protected members are not accessible in derived classes.
- ☑ public methods often form the visible interface of a class.

Ex. Objects and the “this” pointer
Choose one or more options
- ☑ An object is an instance of a class.
- ☑ Each object of a given class has its own copy of the non-static data members.
- ☑ Member functions are typically shared among all objects of a class.
- ☑ Inside non-static member functions, “this” is a pointer to the current object.
- ☐ The “this” pointer can be changed to point to another object.

Ex. Inheritance
Choose one or more options
- ☑ Inheritance allows creating new classes based on existing ones.
- ☐ A derived class can only change members of the base class.
- ☑ Public inheritance models an “is-a” relationship.
- ☑ Multiple inheritance allows a class to inherit from more than one base class.
- ☐ Inheritance is the only way to reuse code in C++.

Ex. Constructors and destructors
Choose one or more options
- ☑ A constructor is a special member function that initializes an object.
- ☑ A default constructor takes no parameters.
- ☐ The user must never create a constructor as the compiler always generates one implicitly.
- ☐ A destructor must be called explicitly when an object is destroyed.
- ☐ There can be multiple destructors with different parameter lists in a class.

Ex. Resource-managing classes (simple exampl
Choose one or more options
- ☑ A class that allocates dynamic memory should typically release it in its destructor.
- ☑ Failing to free dynamically allocated memory in a destructor may cause memory leaks.
- ☑ A constructor that allocates memory can generate an exception if allocation fails.
- ☑ A destructor is responsible for cleaning up internal dynamic resources.
- ☐ If a class only contains automatic (stack) objects, a custom destructor is always required.